Multigrained Angle Representation for Remote-Sensing Object Detection

Hao Wang, Zhanchao Huang, *Graduate Student Member, IEEE*, Zhengchao Chen, Ying Song, and Wei Li, *Senior Member, IEEE*

***Abstract*— Arbitrary-oriented object detection (AOOD) plays a significant role in image understanding in remote-sensing scenarios. The existing AOOD methods face the challenges of ambiguity and high costs in angle representation. To this end, a multigrained angle representation (MGAR) method, consisting of coarse-grained angle classification (CAC) and fine-grained angle regression (FAR), is proposed. Specifically, the designed CAC avoids the ambiguity of angle prediction by discrete angular encoding (DAE) and reduces complexity by coarsening the granularity of DAE. Based on CAC, FAR is developed to refine the angle prediction with much lower costs than narrowing the granularity of DAE. Furthermore, an Intersection over Union (IoU)-aware FAR-Loss (IFL) is designed to improve the accuracy of angle prediction using an adaptive reweighting mechanism guided by IoU. Extensive experiments are performed on several public remote-sensing datasets, which demonstrate the effectiveness of the proposed MGAR. Moreover, experiments on embedded devices demonstrate that the proposed MGAR is also friendly for lightweight deployments.**

***Index Terms*— Angle representation, arbitrary-oriented object detection (AOOD), lightweight model, remote-sensing image.**

I. Introduction

IN RECENT years, arbitrary-oriented object detection (AOOD) has been widely used in complex remote sensing, aviation, and other scenes [1], [2], [3]. The oriented bounding boxes (OBBs) provide richer angle information than the horizontal bounding boxes (HBBs), which helps to better localize objects at arbitrary angles in some particular scenes, such as ships arranged densely in the harbor, as shown in Fig. 1(a).

AOOD methods are developed from classical object detection algorithms, such as Faster region convolutional neural network (R-CNN) [4] and RetinaNet [5]. The mainstream AOOD methods adopt five parameters to describe the position information of the OBB. They are the coordinates (x, y) of the OBB's center and OBB's width w, height h, and rotation angle θ. Most of the existing AOOD methods directly predict the rotation angle by regression. In the beginning, AOOD methods mainly improve the network structure to adapt to rotating objects [6], [7], [8], and some Anchor-based methods [9], [10] used rotated anchors rather than horizontal anchors to obtain more suitable regional proposals. Region-of-Interest (RoI) transformer [11] learns oriented proposals from horizontal RoI, and oriented R-CNN [12] designs oriented region proposal network (RPN) to represent rotating proposals. They all start with improving the method of proposal generation to obtain higher-quality proposals. However, on the one hand, they may face ambiguous predictions because of the periodicity of angles. For instance, as shown in Fig. 1(b), the two angles of 1.4° and 175.3° are visually consistent, but the regression losses are different, which causes a sudden change in the loss function and further affects model learning. On the other hand, the extensive regression range from 0° to 180° presents instability for the angle regression. Although methods such as Intersection over Union (IoU)-Smooth L1 loss [13] and Modulated loss [7] avoid loss mutation by adding constraints to the regression loss function to learn the angle representation better, they do not consider the problem from the fundamental perspective of angle regression.

Circular smooth label (CSL) [14] and densely coded labels (DCL) [15] methods consider things from a new perspective, and they transform the angle regression to a discrete classification problem by converting continuous angle information to discrete angular encoding (DAE). Although the CSL method resolves the ambiguity by fine-grained angle classification (FAC), which divides the angle into a category at

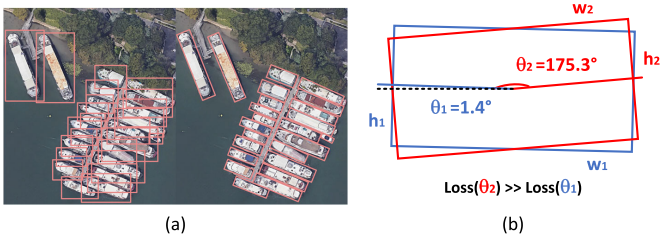


Fig. 1. (a) Different forms of annotation. (Left) HBBs. (Right) OBBs. (b) Mutation of regression loss.

Manuscript received 9 July 2022; revised 5 September 2022; accepted 4 October 2022. Date of publication 5 October 2022; date of current version 20 October 2022. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB3901103 and in part by the Aeronautical Science Foundation of China under Grant 2020051072001. *(Corresponding authors: Ying Song; Wei Li.)*

Hao Wang and Zhanchao Huang are with the School of Information and Electronics and the Beijing Key Laboratory of Fractional Signals and Systems, Beijing Institute of Technology, Beijing 100081, China (e-mail: haohaolalahao@icloud.com; zhanchao.h@outlook.com).

Zhengchao Chen is with the Aerospace Information Research Institute, Chinese Academy of Sciences, Beijing 100094, China (e-mail: chenzc@radi.ac.cn).

Ying Song is with the School of Information and Communication Engineering, Hubei University of Economics, Wuhan 430205, China (e-mail: prisong@163.com).

Wei Li is with the School of Information and Electronics, Beijing Institute of Technology, Beijing 100811, China, and also with the Luoyang Institute of Electro-Optical Equipment, Aviation Industry Corporation of China, Ltd., Luoyang 471000, China (e-mail: liwei089@ieee.org).

Digital Object Identifier 10.1109/TGRS.2022.3212592

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

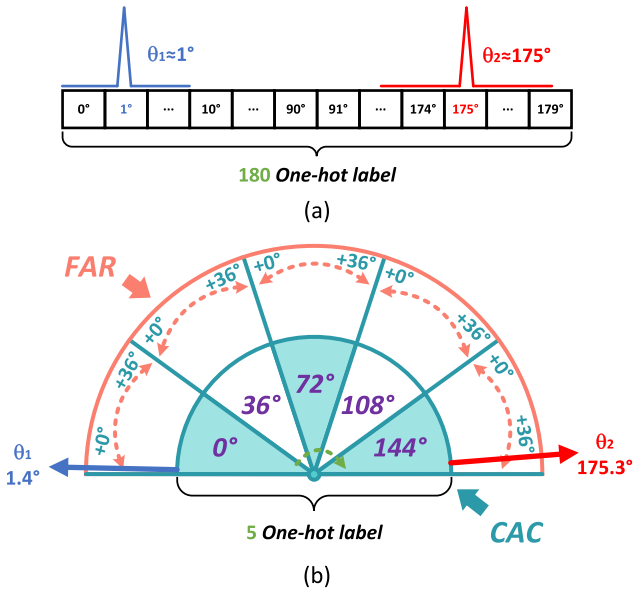


Fig. 2. Angle representation of the perspective based on the classification method. (a) Heavy prediction layers. (b) Proposed MGAR.

1° intervals in 0°–180°, as shown in Fig. 2(a), it still faces the following issues. First, an additional hyperparameter, that is, window size, is introduced by CSL to smooth one-hot encoding labels, but it is sensitive to different datasets. In addition, FAC increases the number of prediction layers of the CNN model, which affects the model's efficiency. In this regard, DCL improves CSL by introducing Binary-Code and Gray-Code into FAC to obtain sparse encoding labels and reduce the CNN prediction layers. However, encoding errors still exist, and the encoding length depends on a hyperparameter sensitive to different datasets. In addition, the introduced label decoding also affects the inference speed of the CNN model.

To solve the above issues, a multigrained angle representation (MGAR) method is proposed. The proposed MGAR divides the angle representation into two parts, as shown in Fig. 2(b). The first part is coarse-grained angle classification (CAC), and the second part is fine-grained angle regression (FAR). The CAC first divides the angle into multiple coarse categories and determines which category the angle belongs to. The FAR performs a fine continuous regression on a small range of angle classification categories. Except for this, we design an IoU-aware FAR-Loss (IFL) for FAR to improve accuracy by using an adaptive reweighting mechanism guided by IoU. The main contributions of this article are summarized as follows.

1) The proposed MGAR includes CAC and FAR. The designed CAC refrains from the ambiguity of angle prediction and increases model calculation efficiency by reducing angle categories. In addition, the CAC is conducive to the learning of the classification task without introducing the hyperparameter sensitive to different datasets.
2) Based on the CAC, the designed FAR makes the angle prediction more refined, and meanwhile, the encoding error is avoided, further improving the computing efficiency. Furthermore, the IFL makes the angle regression smoother and more stable at convergence, achieving extremely high performance.
3) Experiments are implemented on five public large-scale remote-sensing and aerial scene datasets. The results demonstrate the effectiveness of the proposed method in terms of accuracy and speed. Furthermore, the advantages of hyperparameter insensitivity and performance stability of the proposed MGAR are evaluated. The proposed MGAR can avoid hyperparameter selection and save training costs. Besides, the scalability of MGAR for lightweight deployment on embedding devices is analyzed. The proposed MGAR improves the performance of the lightweight AOOD model and maintains a fast detection speed under power-constrained conditions.

The rest of this article is organized as follows. Section II introduces the related work and the existing angle representation methods for the AOOD method. Section III introduces the proposed method MGAR. Section IV analyzes the performance of the proposed MGAR through extensive experiments. Section V draws conclusions.

II. Related Works

A. Arbitrary-Oriented Object Detection

As a fundamental task of computer vision, object detection methods are widely used in natural scene detection [16], face recognition [17], and other fields [18], [19]. Current CNN-based object detection methods are divided into two-stage methods and single-stage methods from the network structure. The R-CNN series [4], [20] accelerate the development of two-stage methods but are limited by the detection speed and model size. Therefore, single-stage methods that focus more on speed are proposed, such as RetinaNet [5] and YOLO series [21], [22], [23]. In recent years, object detection methods are different from the past Anchor-based methods, and a series of Anchor-free methods have emerged successively, such as fully convolutional one-stage (FCOS) object detection [24] and CornerNet [25]. AOOD methods are mainly derived from object detection methods. Some AOOD methods have been developed to deal with the challenges of various angle information in remote sensing, aviation, and other special scenes. The two-stage method small, cluttered and rotated objects detection (SCRDET) [13] introduces the attention module to eliminate the noise problem caused by complex remote-sensing scenes. The single-stage method refined rotation RetinaNet detection (R3Det) [8] designs a feature refinement module based on RetinaNet to further refine the OBB. Anchor-free method general Gaussian heatmap label (GGHL) [26] further refines the features of the rotating object by introducing the 2-D Gaussian label distribution based on FCOS. In addition, there is also a method named lightweight oriented object detection (LO-Det) [27] for lightweight research on AOOD.

B. Angle Representation of OBB

The definition of the OBB in existing AOOD methods is divided into two categories: the five-parameter method and the eight-parameter method, where the five-parameter method is divided into the 90° definition method based on

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

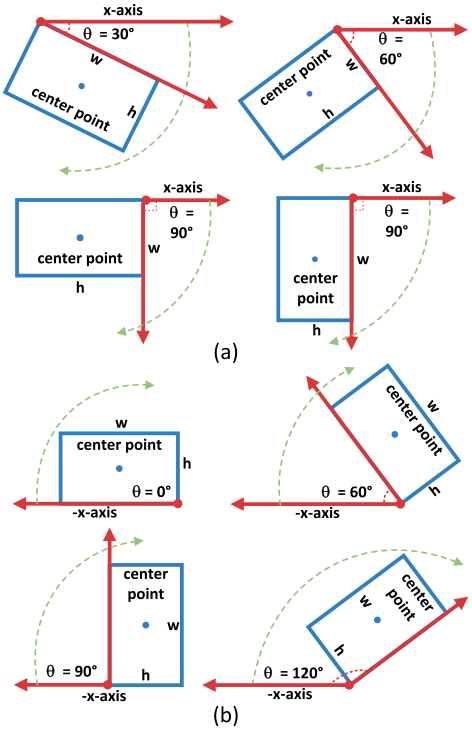


Fig. 3. Two five-parameter methods. (a) OpenCV-based method with 90° angle range. (b) Long-side-based method with 180°.

the OpenCV and the 180° definition method extended on this definition. There are many problems in using the regression-based method to predict angles directly. Some methods solve the problems in OBB regression from the loss function, such as SkewIOU [9], which is optimized for the large aspect ratio problem. Some methods, such as Gaussian Wasserstein Distance (GWD) [28], convert the five-parameter method to a 2-D Gaussian distribution representation and design a novel loss function to regress the OBB indirectly. Other methods such as CSL and DCL utilize the idea of classification to predict angles. The eight-parameter method uses the four-coordinate position representation of the OBB. However, there is a vertex sorting problem. Gliding Vertex [29] avoids sorting by changing the representation of the bounding box, while RSDet [7] designs corner points sorting algorithm to achieve OBB prediction.

The current AOOD methods have different definitions for the five-parameter method. For the OpenCV, as shown in Fig. 3(a), the θ is defined as the angle between the positive x-axis and the first side it encounters when rotating clockwise, and the angle ranges from $(0°, 90°]$. In this definition, there is a possibility of exchange between the two sides of the OBB, and the angle range varies periodically. These two problems lead to abrupt changes and discontinuities in the model's loss function during training. To avoid the problems caused by the OpenCV-based five-parameter method, we adopt the 180° definition method based on the long side. It is displayed in Fig. 3(b), where the longest side is w and the shortest side is h. Furthermore, the angle is defined as the angle between the long side of the OBB and the negative x-axis, and the angle range is $[0°, 180°)$.

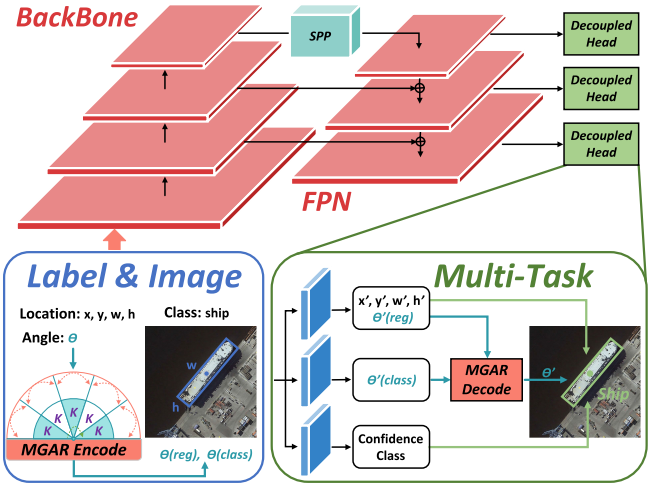


Fig. 4. Baseline framework architecture for the MGAR method. Including three parts: BackBone, FPN, and Decoupled Head. The decoupled head contains three multitask branches which are the location regression branch, angle classification branch, and object classification branch.

III. Proposed Detection Methodology

Fig. 4 illustrates the baseline framework for the proposed MGAR, including BackBone, feature pyramid networks (FPNs) [30], and Decoupled Head. Then the proposed MGAR method is compared with the regression-based, CSL, and DCL methods in four aspects: representation, error analysis, prediction layer thickness, and hyperparameters. The loss function is introduced in the end.

A. Baseline Framework

Compared to the two-stage detection methods, single-stage detection methods achieve a better balance between speed and accuracy. They are more convenient when designing a lighter model for practical deployment. Therefore, we adopt the widely used single-stage method YOLO as the baseline and improved it for the AOOD task. To avoid the impact of complex networks on the performance of the proposed MGAR, we choose the YOLOv3 [23] model with a more concise network structure to adapt to the AOOD better and use this model as the baseline. The mainstream single-stage models usually consist of three parts: backbone, neck, and head. Our baseline model improved from YOLO utilizes Darknet53 [23] as the backbone and FPN as the neck and adopts a highly coupled detection head. We add the spatial pyramid pooling (SPP) [31] structure between the backbone and FPN to achieve better feature fusion. The coupled head is transformed into a decoupled head to avoid the impact of an overly coupled prediction layer on different prediction parameters. In addition, the output of each FPN layer is decomposed into three multi-task prediction branches: regression branch, class classification branch, and angle classification branch. The regression branch is responsible for regressing the center coordinates of the OBB, the long side w, the short side h, as well as the regression part of the angle information. The class classification branch predicts the category to which the object belongs and distinguishes the confidence score between the foreground and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

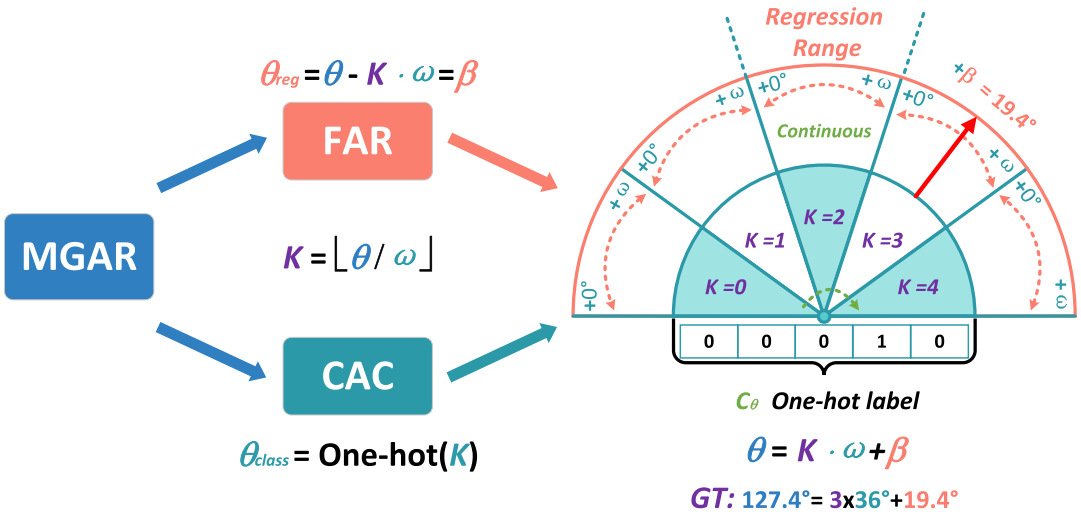


Fig. 5. Proposed MGAR is divided into two parts: the CAC and the FAR. The CAC encodes the angle classification label and the FAR encodes the angle regression label. A specific example is illustrated.

the background. The angle classification branch is employed to predict the class of the angle information.

B. Multigrained Angle Representation

The proposed MGAR divides the angle information into two parts for representation. One is the coarse-grained classification part denoted as θ_{class}; the other is the fine-grained regression part expressed as $\theta_{\text{regression}}$. For the prediction of the network, the proposed MGAR has encoding and decoding processes. We use θ_{gt} to denote the true angle label in the range of $[0°, 180°)$, $\theta_{\text{encodeClass}}$ to denote the angle encoding classification label, $\theta_{\text{encodeRegression}}$ to indicate the angle encoding regression label, $t'_{\theta_{\text{class}}}$ to denote the prediction result of the angle classification part of the network, and $t'_{\theta_{\text{reg.}}}$ to denote the prediction of the angle regression part of the network. Correspondingly, the decoding information is represented by $\theta_{\text{decodeClass}}$ and $\theta_{\text{decodeRegression}}$. The final angle prediction result is expressed by θ'. The specific encoding and decoding processes are represented as follows:

$$k = \left\lfloor \frac{\theta_{gt}}{\omega} \right\rfloor \tag{1}$$

$$\theta_{\text{encodeClass}} = \text{Onehot}(k) \tag{2}$$

$$\theta_{\text{encodeRegression}} = \theta_{gt} - k \times \omega \tag{3}$$

$$\theta_{\text{decodeClass}} = \omega \times \text{Argmax}\left(\text{Sigmoid}\left(t'_{\theta_{\text{class}}}\right)\right) \tag{4}$$

$$\theta_{\text{decodeRegression}} = \left(t'_{\theta_{\text{reg.}}}\right)^2 \tag{5}$$

$$\theta' = \theta_{\text{decodeClass}} + \theta_{\text{decodeRegression}} \tag{6}$$

where $\omega = \text{AR}/C_\theta$ denotes the discretization granularity of the angle classification part. AR represents the angle range, which is 180°. C_θ indicates the number of coarse-grained categories that the angle needs to be divided into. k represents the specific category of the angle. We use square function to fitting $t'_{\theta_{\text{reg.}}}$ which is smoother than linear function. The fine-grained regression range of the angle is $[0°, \omega)$. The detailed MGAR is displayed in Fig. 5.

C. Coarse-Grained Angle Classification

In the CAC, when $C_\theta = 1$, the number of angle classifications is only one category, and the angle regression range is $[0°, 180°)$. At this time, MGAR is converted to a regression-based method. When $C_\theta = 180$, the angle regression range is $[0°, 1°)$, and the angle classification part is equivalent to the classification-based angle encoding method CSL. The value range of C_θ is $[1, 180]$ theoretically, but two problems need to be considered in practical applications. The first problem is that when C_θ is not divisible by 180, ω is not an integer, which makes the angle classification branch produce an unavoidable floating-point calculation error when decoding. In order to avoid this error, C_θ should be the number of categories that can be divided by 180. The second problem is that when the classification granularity is too fine, such as in the case of $C_\theta = 180$, there are too many classification categories, increasing the difficulty of model training. CSL introduces the Gaussian window function based on the one-hot label, converting the hard label to a soft label. Although CSL overcomes the hindrance of FAC in learning, it also introduces a dataset-sensitive hyperparameter window size, which needs to be adjusted for different datasets. Our method utilizes CAC. Specifically, the introduced hyperparameter C_θ is insensitive to the datasets and $C_\theta \in [3, 4, 5]$. When the number of categories is small, the one-hot label can be used to obtain a good enough classification result without adding additional window functions.

The DCL method reduces the length of angle classification labels through binary encoding and gray encoding. However, the introduced hyperparameter C_θ is sensitive to the dataset, and the encoding error cannot be ignored. The maximum encoding error of the two encoding methods is represented as follows:

$$\text{MAX(error)} = \frac{\omega}{2} = \frac{90}{C_\theta}. \tag{7}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

TABLE I

COMPARISON OF FOUR ANGLE REPRESENTATION METHODS, INCLUDING MAXIMUM ENCODING ERROR, AVERAGE ENCODING ERROR, INTRODUCED HYPERPARAMETER, AND ITS SENSITIVITY

Methods	C_θ	Maximum Encoding Error	Average Encoding Error	Hyperparameter	Sensibility
Reg.	1	0	0	-	-
CSL	180	0.5	0.25	Window Size	yes
DCL	256	0.3515625	0.17578125	C_θ	yes
MGAR	3	0	0	C_θ	No

TABLE II

COMPARISON OF THREE METHODS FOR PREDICTION LAYER THICKNESS, FLOPS(G), AND PARAMETERS(M), UNDER THE SAME BASELINE FRAMEWORK

Methods	C_θ	Anchor	Thickness	FLOPs(G)	Parameters(M)
baseline+CSL	180	9	1620	141.8064	75.9356
baseline+DCL	32	9	45	139.4475	75.9356
baseline+MGAR	3	9	**36**	**139.4340**	**74.9878**

Note: The unit G is Giga, which represents 1×10^9. The unit M represents 1×10^6.

The average encoding error is represented as follows:

$$E(\text{error}) = \int_a^b \frac{x}{b-a}dx = \int_0^{\frac{\omega}{2}} \frac{x}{\frac{\omega}{2}-0}dx = \frac{\omega}{4} = \frac{45}{C_\theta}. \quad (8)$$

The specific error profiles of the methods are listed in Table I. The proposed MGAR does not produce errors when encoding angles. Specifically, MGAR encodes continuous angle information into the regression part, avoiding the different errors caused by the angle encoding method based only on the classification method when encoding θ_{gt}.

D. Fine-Grained Angle Regression

The FAR of MGAR relies on the CAC to reduce the range from the original [0°, 180°) to [0°, (180°/C_θ)). Narrowing the angle regression range significantly improves detection accuracy and reduces the volatility of the regression. Meanwhile, the regression value range is continuous, which regresses the angle information more finely while avoiding the problem of loss mutation. Furthermore, it fits the OBB more accurately than the classification-based method only.

The thickness of the network's final prediction layer is inconsistent under different angle representation methods. We use Th to denote the prediction layer thickness and A to represent the number of anchors. For CSL, the prediction layer thickness is expressed as follows:

$$\text{Th}_{\text{CSL}} = A \times \text{AR}/\omega = A \times C_\theta \quad (9)$$

where $C_\theta = 180$.

For DCL, the prediction layer thickness is expressed as follows:

$$\text{Th}_{\text{DCL}} = A \times \left[\log_2(\text{AR}/\omega)\right] = A \times \left[\log_2(C_\theta)\right] \quad (10)$$

where $C_\theta \in [32, 64, 128, 256]$.

For the proposed MGAR, the prediction layer thickness is expressed as follows:

$$\text{Th}_{\text{MGAR}} = A \times (\text{AR}/\omega + 1) = A \times (C_\theta + 1) \quad (11)$$

where $C_\theta \in [3, 4, 5]$.

For the baseline framework, when C_θ taking the minimum value allowed by each method, the prediction layer thickness, floating-point operations (FLOPs), and parameters of several encoding methods are listed in Table II.

The proposed MGAR requires about 97% less prediction layer thickness and about 2.37 fewer FLOPs for angle representation than CSL. The required prediction layer thickness for MGAR is comparable to DCL. Nevertheless, the DCL method introduces binary and gray encoding, resulting in additional time overhead in decoding the angle information. Overall, the proposed MGAR method helps to reduce the computational effort and improve the computational efficiency of the model, especially in lightweight deployment.

E. Loss Function

For the proposed method, we use six parameters (x, y, w, h, θ_{class}, $\theta_{\text{reg.}}$) to represent the OBB, where (θ_{class}, $\theta_{\text{reg.}}$) represents the two parts of θ, (x, y) denotes the relative coordinates of the center point of the OBB, and (w, h) corresponds to the long and short sides of the OBB, respectively. For the other five parameters to be regressed, the regression equations are indicated as follows:

$$t_x = (x - x_a)/w_a, \quad t_y = (y - y_a)/h_a \quad (12)$$

$$t_w = \log(w/w_a), \quad t_h = \log(h/h_a) \quad (13)$$

$$t_{\theta_{\text{reg.}}} = \sqrt{\theta_{\text{reg}}} \quad (14)$$

$$t'_x = \left(x' - x_a\right)/w_a, \quad t'_y = \left(y' - y_a\right)/h_a \quad (15)$$

$$t'_w = \log\left(w'/w_a\right), \quad t'_h = \log\left(h'/h_a\right) \quad (16)$$

$$t'_{\theta_{\text{reg.}}} = \sqrt{\theta'_{\text{reg}}} \quad (17)$$

where (x_a, y_a, w_a, h_a) denotes the center coordinates, long side, and short side of the anchor box, respectively. (x, y, w, h, $\theta_{\text{reg.}}$) denotes the ground-truth OBB, and (x', y', w', h', $\theta'_{\text{reg.}}$) represents the final prediction value. ($t_x, t_y, t_w, t_h, t_{\theta_{\text{reg.}}}$) is the final output value of the network.

The multitask loss function consists of five components: location regression, confidence classification, class classification, angle category classification, and angle regression. The specific loss function used in the proposed MGAR is expressed as follows:

$$\begin{aligned} L = {} & \frac{\lambda_1}{N}\sum_{n=1}^{N} \text{obj}_n \times L_{\text{IoU}}\left(\left(x', y', w', h'\right), (x, y, w, h)\right) \\ & + \frac{\lambda_2}{N}\sum_{n=1}^{N} L_{\text{conf}}\left(\text{conf}', \text{conf}\right) + \frac{\lambda_3}{N}\sum_{n=1}^{N} \text{obj}_n \times L_{\text{cls}}(p, t) \\ & + \frac{\lambda_4}{N}\sum_{n=1}^{N} \text{obj}_n \times L_{\text{cls}}\left(\theta'_{\text{class}}, \theta_{\text{class}}\right) \\ & + \frac{\lambda_5}{N}\sum_{n=1}^{N} \text{obj}_n \times L_{\text{reg}}\left(\theta'_{\text{reg.}}, \theta_{\text{reg.}}\right) \times (|-\log(\text{IoU})| + 1) \end{aligned} \quad (18)$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

where hyperparameter $\lambda_i (i = 1, 2, 3, 4, 5)$ controls the weight distribution of different loss components, for all datasets, we set them to $(2, 2, 5, 2, 0.5)$, respectively. The obj_n serves to distinguish whether the assigned label is foreground or background. It represents foreground when $n = 0$ and represents background when $n = 0$. For the four parameters (x, y, w, h), we use GIoU loss [32] for regression. The conf denotes the object confidence, which predicts the probability that the objects belong to the foreground, and we use Focal loss [5] to calculate this loss. For the class of the objects, we regard p as the predicted class probability and t as the true class, and we adopt the cross-entropy function to calculate the class loss. The same cross-entropy is used for the coarse-grained classification of the angles to learn the encoding angle category information. For the angle regression, we design an IFL, which introduces the object's IoU score based on smooth L1 loss [4] and uses $|-\log(\text{IoU})|+1$ to adaptively reweighting the loss. IFL can guide angle regression more smoothly.

IV. Experiments and Analysis

In this section, experiments on public remote-sensing datasets are conducted to verify the effectiveness of the proposed MGAR. The hardware and software platforms, implementation details, and experimental datasets are presented first. Then ablation experiments are performed, and comparative experiments and discussions are carried out on several public datasets.

All the experiments are implemented on a computer with an AMD EPYC 7542@2.9-GHz CPU, 128 GB of memory, and two GPUs of NVIDIA GeForce RTX 3090 24 GB. In addition, to verify that the proposed method is conducive to lightweight deployment, we also test it on the embedded device NVIDIA Jetson AGX Xavier. We use Pytorch [33] to implement the framework.

A. Experimental Datasets

1) HRSC2016: HRSC2016 [34] is an oriented ship detection dataset, including two main scenes of nearshore and offshore, with 2976 ship targets. The image size varies from 300×300 pixels to 1500×900 pixels. The dataset is divided into a training set with 436 images, a validation set with 181 images, and a testing set with 444 images.

2) DOSR: DOSR [35] is a dataset for oriented ship recognition. The DOSR dataset is mainly collected from Google Earth, including 1066 optical remote-sensing images and 6127 ship instances. The image size is ranged from 600 to 1300 pixels with resolutions of 0.5–2.5 m. The dataset contains rich scenes, including nearshore scenes and offshore scenes. And the dataset contains 20 fine-grained classes of ships, which are Submarine (Sub), Tanker (Tan), Bulk Cargo Vessel (BCV), Auxiliary Ship (Aux), Yacht (Yac), Military Ship (Mil), Barge (Bar), Flat Traffic Ship (FTS), Deck Barge (DeB), Cruise (Cru), Container (Con), Cargo (Car), Transport (Tra), Deck Ship (DeS), Floating Crane (Flo), Fishing Boat (Fis), Tug, Communication Ship (Com), Multihull (Mul), and Speedboat (Spe). The object distribution of this dataset belongs to the long-tail distribution.

3) UCAS-AOD: UCAS-AOD [36] contains 1510 aerial images, including two categories of airplanes and cars, with 14 596 instances. The sizes of images are mostly 659×1280 pixels. According to the usual division manner, the UCAS-AOD dataset is divided into a training set with 755 images, a validation set with 302 images, and a testing set with 452 images.

4) DIOR-R: The DIOR-R [37] dataset is based on the DIOR dataset with OBB-based annotations added. The dataset consists of 23 463 images and 190 288 object instances with 20 classes. The size of all images are 800×800 pixels.

5) DOTA: The DOTA [1] dataset is a classic remote-sensing scene rotating object dataset, which contains 2806 huge remote-sensing images ranging from 800 to 4000 pixels. There are 15 classes in DOTA, including Plane (PL), Baseball diamond (BD), Bridge (BR), Ground field track (GFT), Small vehicle (SV), Large vehicle (SV), Ship (SH), Tennis court (TC), Basketball court (BC), Storage tank (ST), Soccer-ball field (SBF), Roundabout (RA), Harbor (HA), Swimming pool (SP), and Helicopter (HC).

B. Evaluation Metrics and Implementation Details

1) Evaluation Metrics: For AOOD methods, the mainstream evaluation metric is the mean Average Precision (mAP), which is consistent with HBB, and IoU is calculated by rotating IoU with angles. We use two mAP calculation methods based on PASCAL VOC: VOC07 [38] and VOC12 [39]. Usually, mAP is calculated according to an IoU threshold of 0.5. In some cases where we need to compare higher accuracy, the results of mAP calculations at IoU thresholds of 0.75 and 0.85 are used. We use the detected frames per second (FPS) to evaluate the speed of detection. The FLOPs are used to evaluate the theoretical computational complexity of a model. And the number of parameters is used to evaluate the model size.

2) Implementation Details: All datasets are trained with training and validation sets. The input image size is 800×800 pixels for the HRSC2016, UCAS-AOD, and DIOR-R datasets. The network input size for the DOSR dataset is set to 1024×1024 pixels because of its large average image size and dense scenes. For the DOTA dataset, because the original image size is large, the original image is cropped by 800×800 pixels with a stride of 200 pixels. In addition, the zoom scale is [0.5, 1.0, 1.5] when cropping, and images with the size of 896×896 pixels are used as network input for both training and testing. The data augmentation strategy is adopted to alleviate the overfitting problem, including horizontal and vertical flipping, the random color transformation of hue, saturation, value (HSV) color gamut, mixup [49], and random rotation in three directions of 90°, 180°, and 270°. For batch size, it is 16 when the input image size is 800×800 pixels and 8 when the input image size is 1024×1024 pixels. The stochastic gradient descent (SGD) optimizer with a momentum of 0.9 and a weight decay of 1×10^{-5} is used during training. The initial learning rate is set to 1×10^{-4}, and the final learning rate is 1×10^{-5}.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

TABLE III
ABLATION STUDIES EVALUATED UNDER mAP (%) (VOC12) ON THE HRSC2016 DATASET

Methods	SPP	C_θ	mAP_{50}	mAP_{85}	$mAP_{50:95}$	Speed (FPS)
Regression		1	91.42	11.57	49.99	51.73
Reg. (baseline)	✓	1	92.02	14.97	52.86	51.44
baseline + CSL	✓	180	97.41 (+5.39)	43.72 (+28.75)	68.35 (+15.49)	50.53 (-0.91)
baseline + DCL(binary)	✓	128	93.86 (+1.84)	18.06 (+3.09)	56.23 (+3.37)	52.63 (+1.19)
baseline + DCL(gray)	✓	64	97.44 (+5.42)	42.09 (+27.12)	68.18 (+15.32)	53.24 (+1.80)
baseline + MGAR	✓	5	**97.62 (+5.60)**	**49.58 (+34.61)**	**68.83 (+15.97)**	**56.21 (+4.77)**

Note: Speed is the test result on NVIDIA GeForce RTX 3090. The speed (average of 10 tests) includes the network inference speed with post-processing. The input size of image for network is 800×800 pixels.

TABLE IV
COMPARISON OF HYPERPARAMETER'S SENSITIVITY UNDER mAP (%) (VOC12) ON THE HRSC2016 DATASET

Methods	C_θ	mAP_{50}	mAP_{85}	$mAP_{50:95}$
baseline + DCL(binary)	32	94.15	13.97	53.28
	64	90.10	7.95	46.80
	128	93.86	18.06	56.23
	256	93.67	15.34	54.6
	$\mu \pm \sigma$	93.84±1.91	19.81±4.27	55.85±4.13
baseline + DCL(gray)	32	97.02	32.13	65.65
	64	97.44	42.09	68.18
	128	97.26	34.54	65.91
	256	97.28	24.28	62.32
	$\mu \pm \sigma$	97.25±0.15	33.26±6.35	65.52±2.09
baseline + MGAR	5	97.62	49.58	68.83
	4	97.19	48.88	68.77
	3	97.46	48.66	68.66
	$\mu \pm \sigma$	**97.42±0.17**	**49.04±0.39**	**68.75±0.07**

Note: μ represents Average, and σ represents Standard Deviation

C. Ablation Study

In the case of the same angle deviation, objects with larger aspect ratios cause greater deviation in calculating IoU, which further affects the mAP result. The HRSC2016 dataset contains many ship objects with larger aspect ratios, which helps to compare the fitting accuracy of the OBB. Therefore, the HRSC2016 dataset is chosen to conduct ablation experiments, comparing regression, CSL, DCL, and the proposed MGAR. We take the regression-based method as the baseline and implement the other three methods on the basis of it. These four methods are consistent with the experimental settings except for their hyperparameters. For the CSL, the optimal window size of the Gaussian function is 6. DCL contains two types of encoding: binary encoding and gray encoding. When the hyperparameter $C_\theta \in [32, 64, 128, 256]$, DCL has the best performance. For the proposed MGAR, we choose $C_\theta \in [3, 4, 5]$ for comparison.

1) Comparison of Four Methods: The best results of these four methods are listed in Table III. Three accuracy metrics are adopted to evaluate the performance of these four methods. The mAP_{50} is a commonly used base metric, mAP_{85} evaluates the accuracy with more stringent criteria, and $mAP_{50:95}$ aims to evaluate the overall performance of the methods in a more comprehensive way. We compare the differences in model performance before and after the addition of the SPP module on a regression-based method, and from the results, SPP helps to improve accuracy. Compared to other methods, the proposed method MGAR performs best, with mAP_{50}, mAP_{85}, and $mAP_{50:95}$ reaching 62%, 49.58%, and 68.83%, respectively. In particular, mAP_{85} is higher than baseline, CSL, DCL (binary), and DCL (gray), respectively, by 34.61%, 5.86%, 31.52%, and 7.49%. which shows that FAR can get fine-grained angle information more accurately. In addition, for the speed metric, because CSL, DCL, and MGAR all have additional decoding operations, which increase the time overhead in the nonmaximum suppression (NMS) stage, the time listed in Table III includes network inference time with postprocessing. It is observed that our proposed method is the fastest, reaching 56.21 FPS, which is beneficial for lightweight deployment. In summary, the proposed MGAR has an advantage over other methods in terms of speed and accuracy.

We also analyze the sensitivity of hyperparameters C_θ in different methods. Table IV lists the results of DCL (gray), DCL (binary), and the proposed MGAR in the respective optimal hyperparameter range. The results show that different values of C_θ have a large impact on the DCL. It is seen that the proposed MGAR has the best average metric results with the smallest standard deviation, which indicates that the hyperparameters of the proposed MGAR are insensitive. In contrast, the two DCL methods are less stable and more sensitive to C_θ, which implicitly increases the cost of training time on different datasets.

MGAR develops a method based on coarse-grained classification and fine-grained regression, which effectively combines the flexibility of classification and the accuracy of regression, reducing the learning difficulty of FAC and large-scale regression. Fig. 6 visualizes the detection results of four state-of-the-art methods in complex scenes, and it can be observed that the proposed MGAR has a better performance in detail.

2) Impact of Coarse-Grained Angle Classification : Hyperparameter C_θ mainly affects the classification granularity of the angle. For the proposed MGAR, the value of C_θ theoretically determines the thickness of the detection head, which in turn affects the speed of the model. To more fully analyze the impact of C_θ on the MGAR method, we analyze the results for C_θ at larger values. Considering the floating-point error caused by C_θ not being divisible by 180°, we discuss the cases of $C_\theta = [6, 9, 10, 12, 15, 18, 20, 30, 45, 60, 90]$. Table V presents the experimental results, and the influence of C_θ on the three accuracy metrics is shown in Fig. 7. It is observed that as C_θ increases, there is a partial increase in mAP_{50}, while mAP_{85} and $mAP_{50:95}$ decrease to some extent. This is mainly because the increase of angle categories increases the difficulty

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

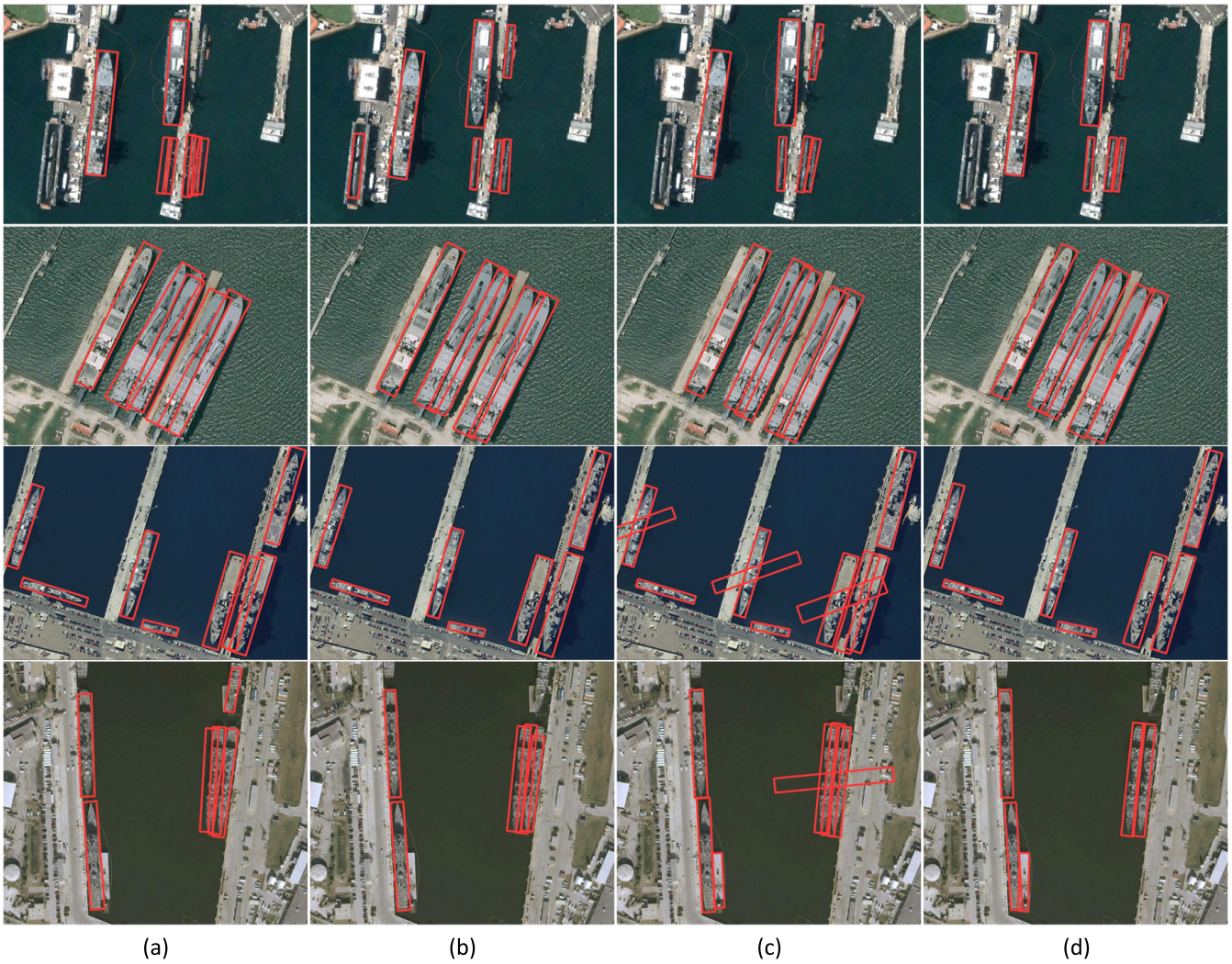


Fig. 6. Comparison of the visualization results of four methods for complex scenarios on the HRSC2016 dataset. (a) Regression-based method. (b) CSL method. (c) DCL (gray) method. (d) Proposed MGAR method.

TABLE V
ABLATION STUDIES FOR LARGER C_θ UNDER mAP (%) (VOC12) ON THE HRSC2016 DATASET

Methods	C_θ	mAP_{50}	mAP_{85}	$\text{mAP}_{50:95}$
baseline + MGAR	6	96.95	47.64	68.57
	9	97.32	46.45	68.12
	10	97.39	43.41	66.45
	12	97.79	38.99	65.82
	15	97.39	18.87	58.16
	18	97.72	22.10	61.14
	20	97.50	22.43	61.69
	30	97.93	29.64	65.46
	45	97.75	39.88	67.29
	60	97.68	37.48	66.90
	90	97.71	37.15	66.48

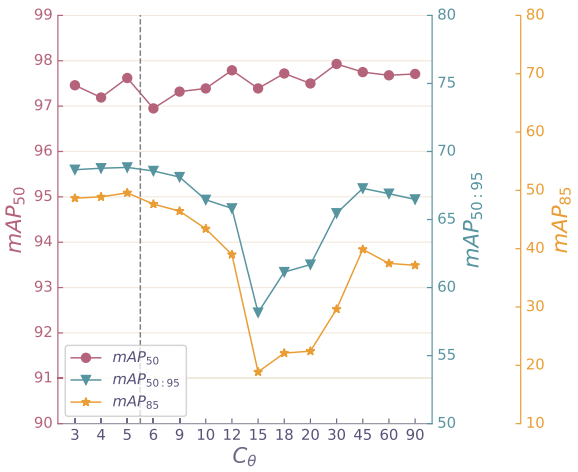


Fig. 7. Trend of the metrics mAP with respect to the hyperparameter C_θ. The red curve represents mAP_{50}, the yellow curve represents mAP_{85}, and the green curve represents $\text{mAP}_{50:95}$.

of network classification. To alleviate this problem, the CSL smooths the one-hot label by introducing the Gaussian window function. However, this method introduces an additional window size hyperparameter, further increasing the parameter selection and invisible training costs. Therefore, to balance accuracy, speed, and parameter stability, the selection range of C_θ is restricted to $[3, 4, 5]$ in the proposed MGAR.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

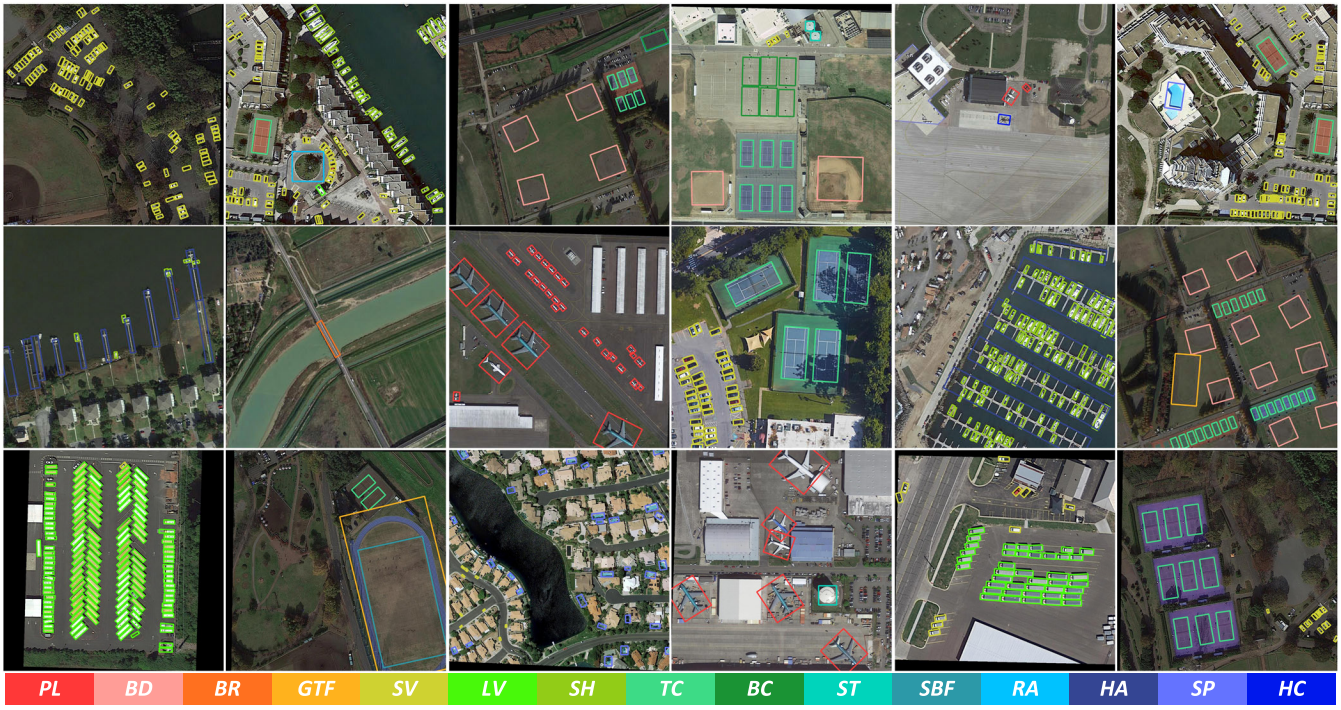


Fig. 8. Visualization results of baseline + MGAR method on the DOTA dataset.

TABLE VI
COMPARISON OF DIFFERENT REGRESSION FITTING FUNCTIONS ON MGAR UNDER mAP (%) (VOC07)

Methods	C_θ	Function	mAP_{50}	mAP_{75}	mAP_{85}	$mAP_{50:95}$
baseline+MGAR	3	*Linear*	97.36	**82.74**	47.29	**68.88**
		Sigmoid	97.18	79.76	43.59	67.14
		Square	**97.46**	82.05	**48.66**	68.66
		Exp	96.78	74.99	38.74	64.56

TABLE VII
COMPARISON OF DIFFERENT REGRESSION LOSS FUNCTIONS ON MGAR UNDER mAP (%) (VOC07)

Methods	C_θ	Loss	mAP_{50}	mAP_{75}	mAP_{85}	$mAP_{50:95}$
baseline+MGAR	3	MSE	93.68	67.74	25.26	58.68
		IFL	**97.46**	**82.05**	**48.66**	**68.66**

3) Impact of Fine-Grained Angle Regression : For angle regression, the proposed MGAR has a clear regression range of $[0°, \omega)$. For the predicted values of the angle regression, $\theta'_{\text{reg.}} = \text{Function}(t'_{\theta_{\text{reg.}}})$ is used to represent the fitting function of the network predict value $t'_{\theta_{\text{reg.}}}$. There are many fitting functions to choose from, such as Linear, Sigmoid, Square, and Exp. Sigmoid function can conform to the angle regression range, and other functions theoretically have the regression range of $[0, +\infty)$. Table VI lists the performance of different fitting functions. Linear and Square functions perform better than Sigmoid and Exp functions, and the Square function is smoother during training than the Linear function. In general, the Square function performs best, so we finally choose it to fit the value of angle regression. For the loss function of angle regression, we compare mean-squared error (MSE) loss

TABLE VIII
COMPARATIVE PERFORMANCE [mAP (%) (VOC07) AND mAP (%) (VOC12)] OF DIFFERENT METHODS ON THE HRSC2016 DATASET

Methods	Backbone	mAP(07)	mAP(12)
R^2CNN [40]	ResNet101 [41]	73.07	79.73
RoI-Transformer [11]	ResNet101	86.20	-
Gliding Vertex [29]	ResNet101	88.20	-
BBAVectors [42]	ResNet101	88.6	-
CenterMap OBB [43]	ResNet50	-	92.8
RetinaNet-R [8]	ResNet101	89.18	95.21
R^3Det [8]	ResNet101	89.26	96.01
R^3Det-DCL [15]	ResNet101	89.46	96.41
S^2ANet [44]	ResNet101	90.17	95.01
Oriented RepPoints [45]	ResNet50	**90.40**	97.26
MGAR($C_\theta = 3$)	DarkNet53	90.32	**97.46**

TABLE IX
COMPARATIVE HIGH-ACCURACY PERFORMANCE [mAP (%) (VOC07)] OF DIFFERENT METHODS ON THE HRSC2016 DATASET

Methods	mAP_{50}	mAP_{75}	mAP_{85}
RetinaNet(GWD) [28]	85.56	60.31	17.14
RetinaNet(KLD) [46]	87.45	72.39	27.68
R3Det(GWD) [28]	89.43	68.88	15.02
R3Det(KLD) [46]	89.97	77.38	25.12
MGAR($C_\theta = 5$)	**90.32**	**79.19**	**46.25**

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

TABLE X

COMPARATIVE PERFORMANCE [AP (%) AND mAP (%) (VOC07)] OF DIFFERENT METHODS ON THE DOSR DATASET. * MEANS FORM [35]

Methods	BCV.	Fis.	DeB.	Yac.	FTS.	Mul.	Tug	Com.	Spe.	Car.	Cru.	Flo.	Tan.	DeS.	Sub.	Con.	Bar.	Tra.	Aux.	Mil.	mAP	Speed (FPS)
FR-FPN-O [35]	37.14	8.79	7.98	47.30	25.59	48.20	48.11	50.80	28.54	80.57	49.81	8.64	65.40	15.54	10.03	56.69	12.21	69.35	10.78	32.18	35.68	4.92
R^2CNN [40]*	56.47	36.86	38.84	57.14	26.19	54.55	58.98	32.03	39.27	76.70	52.99	5.85	50.84	42.41	13.64	75.57	46.08	66.85	43.54	36.85	45.58	2.85
RRPN [9]*	62.89	42.80	33.15	47.15	43.66	52.50	74.57	27.27	31.19	83.30	57.52	33.10	64.89	20.55	54.55	78.61	**52.95**	78.26	36.06	36.63	50.58	2.71
SCRDet [13]*	65.21	54.26	44.62	53.33	49.46	38.31	66.67	40.74	32.19	86.66	61.50	21.95	**87.68**	32.84	54.55	**80.58**	49.91	73.68	42.78	49.46	54.29	3.84
RetinaNet-O [8]*	49.52	26.24	36.86	50.63	27.16	35.00	70.45	49.74	33.68	68.70	35.87	11.93	38.11	2.04	0.00	55.99	12.16	66.27	29.66	34.06	36.70	3.72
R^3Det [8]*	65.60	44.78	28.49	64.21	49.14	53.03	68.16	38.83	35.97	84.24	63.36	34.35	71.63	26.70	57.61	74.77	42.70	74.03	27.19	48.48	52.66	4.89
SCRDet++ [47]*	61.22	44.45	36.06	67.58	62.80	61.62	**81.09**	**66.09**	63.51	76.28	66.14	13.91	77.01	58.98	32.73	70.00	18.75	76.22	**47.27**	44.22	56.33	2.85
RSDet [7]*	55.27	10.76	22.84	59.78	52.56	47.30	58.78	63.64	57.72	75.32	39.60	16.94	33.13	14.23	45.69	68.82	26.19	75.30	33.96	45.04	45.15	2.77
ReDet [6]*	64.28	43.17	27.87	**75.65**	65.06	41.59	80.10	37.67	**67.95**	85.06	62.40	37.54	83.44	36.58	34.85	76.64	44.31	86.03	38.04	57.48	57.32	12.00
EIRNet [35]	67.52	**55.65**	48.59	68.53	**70.48**	57.36	75.52	58.26	42.53	**87.87**	**67.00**	55.30	74.90	56.65	**59.60**	78.63	30.46	78.34	45.98	48.59	61.39	3.68
MGAR†	**68.12**	41.68	**51.98**	71.78	63.64	**81.82**	78.96	63.64	66.22	86.41	59.01	**55.31**	82.39	**63.27**	21.82	75.55	25.97	**88.93**	40.33	**57.65**	**62.22**	**26.17**

Note: The unit FPS means Frames Per Second. To maintain consistency with other methods, we test speed on the same equipment: NVIDIA GeForce GTX 1080Ti. † means $C_\theta = 3$.

and IFL. The results are listed in Table VII. The performance of IFL is better than that of MSE.

D. Comparisons With State-of-the-Arts

1) Results on the HRSC2016 Dataset: We compare other AOOD methods on the HRSC2016 dataset. The experimental results are shown in Table VIII. For convenience, we use "MGAR" to represent "baseline + MGAR" in the following tables. The results indicate that the proposed MGAR has achieved high performance, reaching 90.32% and 97.46% under the metrics mAP(07) and mAP(12), respectively. We also compare the results of different methods under higher detection performance, as listed in Table IX. The proposed method achieves the best results of 79.19% and 46.25% under the two more difficult evaluation metrics of mAP_{75} and mAP_{85}, respectively, which are higher than the Kullback–Leibler Divergence (KLD) [46] and GWD methods.

2) Results on the DOSR Dataset: To further verify the performance of the proposed method, we select a fine-grained long-tailed ship dataset DOSR with more data samples, more complex scenes, and more small objects. This dataset also presents a classification challenge to the AOOD methods. In Table X, we report the AP of each type of ship target. The proposed method is better than other methods, achieving the best result of 62.22% mAP and the fastest speed of 26.17 FPS.

3) Results on the UCAS-AOD Dataset: The UCAS-AOD dataset contains densely arranged small-sized cars and aircraft at different scales, which helps compare the detection effects of methods on small objects. As shown in Table XI, the proposed method can achieve a performance of 90.01% mAP, with excellent performance.

4) Results on the DIOR-R Dataset: DIOR-R contains 20 classes of objects in aviation scenarios. Table XII lists the experimental results on DIOR-R. Our method achieves the best performance of 66.89% mAP.

5) Results on the DOTA Dataset: We utilize the DOTA dataset to evaluate the performance of the proposed method on large datasets. We compare the proposed MGAR with other two-stage methods, refine-stage methods, and single-stage methods. We report the detailed results in Table XIII. Our method obtains 78.29% mAP under the input size of 896 × 896 pixels, which is better than most of the two-stage methods and the refine-stage methods. At the same time, the proposed MGAR achieved the best results in the single-stage methods. It is worth noting that the proposed MGAR performs best in terms of speed, which is the fastest among all comparison methods. We also visualize some detection results

TABLE XI

COMPARATIVE PERFORMANCE [mAP (VOC07)] OF DIFFERENT METHODS ON THE UCAS-AOD DATASET

Methods	Car	Airplane	mAP(07)
YOLOv3-R [23]	74.63	89.52	82.08
RetinaNet-R [8]	84.63	90.51	87.57
Faster R-CNN-R [4]	86.87	89.86	88.36
RoI-Transformer [11]	87.99	89.90	88.95
DAL [48]	89.25	90.49	89.87
S^2ANet [44]	**89.56**	90.42	89.99
MGAR($C_\theta = 5$)	89.40	**90.63**	**90.01**

TABLE XII

COMPARATIVE PERFORMANCE [mAP (VOC07)] OF DIFFERENT METHODS ON THE DIOR-R DATASET

Methods	Backbone	mAP(07)
RetinaNet-O [5]	ResNet-50	57.55
Faster RCNN-O [4]	ResNet-50	59.54
Gliding Vertex [29]	ResNet-50	60.06
RoI-Transformer [11]	ResNet-50	63.87
AOPG [37]	ResNet-50	64.41
Oriented RepPoints [45]	ResNet-50	66.71
MGAR($C_\theta = 5$)	Darknet-53	**66.89**

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII
COMPARATIVE PERFORMANCE [mAP (%) AND SPEED] OF DIFFERENT METHODS ON THE DOTA DATASET

Methods	Stage	Backbone	PL	BD	BR	GTF	SV	LV	SH	TC	BC	ST	SBF	RA	HA	SP	HC	mAP	Speed (FPS)
Faster RCNN-O [1]	Two	R-50	88.44	73.06	44.86	59.09	73.25	71.49	77.11	90.84	78.94	83.90	48.59	62.95	62.18	64.91	56.18	69.05	14.9
ROI-Transformer [11]	Two	R-101	88.53	77.91	37.63	74.08	66.53	62.97	66.57	90.50	79.46	76.75	59.04	56.73	62.54	61.29	55.56	67.74	7.80
SCRDet [13]	Two	R-101	89.98	80.65	52.09	68.36	68.36	60.32	72.41	90.85	87.94	86.86	65.02	66.68	66.25	68.24	65.21	72.61	9.51
Gliding Vertex [29]	Two	R-101	89.64	85.00	52.26	77.34	73.01	73.14	86.82	90.74	79.02	86.81	59.55	**70.91**	72.94	70.86	57.32	75.02	13.10
ReDet [6]	Two	ReR-50	88.79	82.64	53.97	74.00	78.13	84.06	88.04	90.89	87.78	85.75	61.76	60.39	75.96	68.07	63.59	76.25	-
Oriented R-CNN [12]	Two	R-50	89.84	95.43	**61.09**	79.82	79.71	**85.35**	88.82	90.88	86.68	87.73	72.21	70.80	82.42	78.18	**74.11**	**80.87**	8.10
RSDet [7]	Two	R-101	89.80	82.90	48.60	65.20	69.50	70.10	70.20	90.50	85.60	83.40	62.50	63.90	65.60	67.20	68.00	72.20	-
R^3Det [8]	Refine	R-152	89.80	83.77	48.11	66.77	78.76	83.27	87.84	90.82	85.38	85.51	65.67	62.68	67.53	78.56	72.62	76.47	10.53
S^2ANet [44]	Refine	R-50	89.07	82.22	53.63	69.88	80.94	82.12	88.72	90.73	83.77	86.92	63.78	67.86	76.51	73.03	56.60	76.38	17.60
	Refine	R-101	88.89	83.60	57.74	**81.95**	79.94	83.19	**89.11**	90.78	84.87	87.81	70.30	68.25	**78.30**	77.01	69.58	79.42	13.79
Oriented Reppionts [45]	Refine	R-50	87.02	83.17	54.13	71.16	80.18	78.40	87.28	**90.90**	85.97	86.25	59.90	70.49	73.33	72.27	58.97	75.97	16.10
	Refine	R-101	88.86	**88.86**	55.27	76.92	74.27	82.10	87.52	**90.90**	85.56	85.33	65.51	66.82	74.36	70.15	57.28	76.28	14.23
	Refine	Swin-T	88.72	80.56	55.69	75.07	**81.84**	82.40	87.97	90.80	84.33	87.64	62.80	67.91	77.69	**82.94**	65.46	78.12	-
DRN [50]	One	H-104	89.71	82.34	47.22	64.10	76.22	74.43	85.84	90.57	86.18	84.89	57.65	61.93	69.30	69.63	58.48	73.23	-
RIDet [51]	One	R-101	88.93	78.45	46.87	72.63	77.63	80.68	88.18	90.55	81.33	83.61	64.85	63.72	73.09	73.13	56.87	74.70	13.36
PolarDet [52]	One	R-101	89.65	87.07	48.14	70.97	78.53	80.;34	87.45	90.76	85.63	86.87	61.64	70.32	71.92	73.09	67.15	76.64	25.00
GGHL [26]	One	D-53	89.74	85.63	44.50	77.48	76.72	80.45	86.16	90.83	**88.18**	86.25	67.07	69.40	73.38	68.45	70.14	76.95	42.30
GWD [28]	One	R-152	86.96	83.88	54.36	77.53	74.41	68.48	80.34	86.62	83.41	85.55	**73.47**	67.77	72.57	75.76	73.40	76.30	13.86
KFIoU [53]	One	R-152	89.46	85.72	54.94	80.37	77.16	69.23	80.90	90.79	87.79	86.13	73.32	68.11	75.23	71.61	69.49	77.35	13.79
CSL [14]	Two	R-152	**90.25**	85.53	54.64	75.31	70.44	73.51	77.62	90.84	86.15	86.69	69.60	68.04	73.83	71.10	68.93	76.17	8.89
DCL [15]	Refine	R-152	89.26	83.60	53.54	72.76	79.04	82.56	87.31	90.67	86.59	86.98	67.49	66.88	73.29	70.56	69.99	77.37	10.39
MGAR*($C_\theta = 3$)	One	D-53	89.84	85.75	51.59	77.00	76.38	74.81	86.40	90.73	87.70	87.48	63.25	69.70	75.79	80.88	71.07	77.85	**59.17**
MGAR†($C_\theta = 5$)	One	D-53	89.81	85.22	52.51	77.52	77.63	76.19	87.20	90.84	87.93	**88.01**	66.25	67.88	76.24	78.53	72.51	78.29	58.14

Note: The backbone networks R-50, R-101, R-152, ReR-50, H-104, D-53, Swin-T, represent the ResNet50 [41], ResNet101 [41], ResNet152 [41], ReResNet50 [6], Hourglass104 [54], DarkNet53 [23], and Swin Transformer Tiny [55], respectively. "One", "Two", "Refine" represent the one-stage method, two-stage method and refine-stage method, respectively. Speed is the test result on NVIDIA GeForce RTX 3090. The speed (average of 10 tests) only includes the network inference speed without post-processing (batch size = 1). When testing other methods, their open source codes are used. The speed of some methods could not be tested due to the available codes, which is indicated by "-". * means the input size of image for network is 800×800 pixels. † means the input size of image for network is 896×896 pixels.



Fig. 9. Visualization results of the baseline + MGAR method on the DOTA dataset.

on the DOTA testing set, as shown in Fig. 8. The proposed MGAR has excellent detection performance.

E. Lightweight Deployment

To verify the deployment advantages of our method on lightweight devices, we replace the backbone of the baseline with the lightweight CNN MobileNetv2 [56] and test it on the embedded device NVIDIA Jetson AGX Xavier, as shown in Fig. 9. This experiment is conducted on the HRSC2016 dataset. The results are listed in Table XIV. The proposed MGAR achieves the highest accuracy and is less than CSL and DCL in terms of FLOPs and the number of parameters, with theoretically lower computational complexity, less storage consumption, and the fastest speed in the actual test. MGAR is

TABLE XIV
EVALUATION OF DIFFERENT METHODS WITH LIGHTWEIGHT BACKBONE MOBILENETV2 UNDER mAP (%) (VOC12) ON THE HRSC2016 DATASET

Methods	Backbone	mAP_{50}	Speed(FPS)	FLOPs(G)	Param.(M)
baseline+CSL	MobileNetv2	89.54	17.68	9.634	9.21
baseline+DCL*	MobileNetv2	89.43	20.01	8.924	8.42
baseline+MGAR†	MobileNetv2	**89.67**	**21.09**	**8.915**	**8.41**

Note: The unit G is Giga, which represents 1×10^9. The unit M represents 1×10^6. Speed is the speed on NVIDIA Jetson AGX Xavier. The speed (average of 10 tests) includes the network inference speed with post-processing. The input size of image for network is 800×800 pixels. * means $C_\theta = 64$ and use Gray-Code. † means $C_\theta = 3$.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

nearly 3.4 and 1 FPS faster than CSL and DCL, respectively, on the embedded device. Speed and accuracy results demonstrate that the proposed MGAR has excellent advantages in lightweight embedded deployment.

V. Conclusion

In this article, we proposed a novel angle representation method MGAR for AOOD, which combines CAC and FAR. The CAC eliminates the angle ambiguity introduced by previous regression-based methods, reduces the prediction layer thickness, and improves model efficiency. The FAR refines angle prediction, which brings more accurate predictions for objects with large aspect ratios and further reduces computational consumption. Besides, the IFL was designed to help regress angle better and more stably. In particular, the hyperparameter introduced by MGAR is insensitive and robust to different datasets, saving extra training time. We implemented the MGAR method on the improved single-stage method YOLOv3. Extensive experiments on the HRSC2016, DOSR, UCAS-AOD, DIOR-R, and DOTA datasets demonstrated that the proposed method has an excellent performance in both accuracy and speed. We have also conducted experiments on the embedded device, and the results indicated that the proposed MGAR is very friendly for lightweight deployment and has application value.

The code is available at https://github.com/haohaolalahao.

References

[1] G.-S. Xia et al., "DOTA: A large-scale dataset for object detection in aerial images," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3974–3983.

[2] K. Li, G. Wan, G. Cheng, L. Meng, and J. Han, "Object detection in optical remote sensing images: A survey and a new benchmark," *ISPRS J. Photogramm. Remote Sens.*, vol. 159, pp. 296–307, Jan. 2020.

[3] G. Cheng, J. Han, P. Zhou, and D. Xu, "Learning rotation-invariant and Fisher discriminative convolutional neural networks for object detection," *IEEE Trans. Image Process.*, vol. 28, no. 1, pp. 265–278, Jan. 2019.

[4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1–9.

[5] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2980–2988.

[6] J. Han, J. Ding, N. Xue, and G.-S. Xia, "ReDet: A rotation-equivariant detector for aerial object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 2786–2795.

[7] W. Qian, X. Yang, S. Peng, J. Yan, and Y. Guo, "Learning modulated loss for rotated object detection," in *Proc. AAAI Conf. Arti. Intell.*, May 2021, vol. 35, no. 3, pp. 2458–2466.

[8] X. Yang, J. Yan, Z. Feng, and T. He, "R^3Det: Refined single-stage detector with feature refinement for rotating object," in *Proc. AAAI Conf. Artif. Intell.*, May 2021, vol. 35, no. 4, pp. 3163–3171.

[9] J. Ma et al., "Arbitrary-oriented scene text detection via rotation proposals," *IEEE Trans. Multimedia*, vol. 20, no. 11, pp. 3111–3122, Mar. 2018.

[10] Z. Zhang, W. Guo, S. Zhu, and W. Yu, "Toward arbitrary-oriented ship detection with rotated region proposal and discrimination networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 11, pp. 1745–1749, Nov. 2018.

[11] J. Ding, N. Xue, Y. Long, G.-S. Xia, and Q. Lu, "Learning RoI transformer for oriented object detection in aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 2849–2858.

[12] X. Xie, G. Cheng, J. Wang, X. Yao, and J. Han, "Oriented R-CNN for object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 3520–3529.

[13] X. Yang et al., "SCRDet: Towards more robust detection for small, cluttered and rotated objects," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 8232–8241.

[14] X. Yang and J. Yan, "Arbitrary-oriented object detection with circular smooth label," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2020, pp. 677–694.

[15] X. Yang, L. Hou, Y. Zhou, W. Wang, and J. Yan, "Dense label encoding for boundary discontinuity free rotation detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 15819–15829.

[16] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2014, pp. 740–755.

[17] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, "RetinaFace: Single-shot multi-level face localisation in the wild," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 5203–5212.

[18] X. Wu, D. Hong, J. Tian, J. Chanussot, W. Li, and R. Tao, "ORSIm detector: A novel object detection framework in optical remote sensing imagery using spatial-frequency channel features," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 5146–5158, Jul. 2019.

[19] X. Wu, D. Hong, J. Chanussot, Y. Xu, R. Tao, and Y. Wang, "Fourier-based rotation-invariant feature boosting: An efficient framework for geospatial object detection," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 302–306, Feb. 2020.

[20] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 580–587.

[21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.

[22] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 7263–7271.

[23] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.

[24] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 9627–9636.

[25] H. Law and J. Deng, "CornerNet: Detecting objects as paired keypoints," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 734–750.

[26] Z. Huang, W. Li, X.-G. Xia, and R. Tao, "A general Gaussian heatmap label assignment for arbitrary-oriented object detection," *IEEE Trans. Image Process.*, vol. 31, pp. 1895–1910, 2022.

[27] Z. Huang, W. Li, X.-G. Xia, H. Wang, F. Jie, and R. Tao, "LO-Det: Lightweight oriented object detection in remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022.

[28] X. Yang, J. Yan, Q. Ming, W. Wang, X. Zhang, and Q. Tian, "Rethinking rotated object detection with Gaussian Wasserstein distance loss," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 11830–11841.

[29] Y. Xu et al., "Gliding vertex on the horizontal bounding box for multi-oriented object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 4, pp. 1452–1459, Apr. 2021.

[30] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2117–2125.

[31] K. He, X. Zhang, J. Sun, and S. Ren, "Spatial pyramid pooling in deep convolutional networks for visual recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 9, pp. 1904–1916, Jan. 2015.

[32] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2019, pp. 658–666.

[33] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," 2019, *arXiv:1912.01703*.

[34] Z. Liu, L. Yuan, L. Weng, and Y. Yang, "A high resolution optical satellite image dataset for ship recognition and some new baselines," in *Proc. Int. Conf. Pattern Recognit. Appl. Methods*, vol. 2. Setúbal, Portugal: SciTePress, 2017, pp. 324–331.

[35] Y. Han, X. Yang, T. Pu, and Z. Peng, "Fine-grained recognition for oriented ship against complex scenes in optical remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–18, 2021.

[36] H. Zhu, X. Chen, W. Dai, K. Fu, Q. Ye, and J. Jiao, "Orientation robust object detection in aerial images using deep convolutional neural network," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2015, pp. 3735–3739.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.

[37] G. Cheng et al., "Anchor-free oriented proposal generator for object detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–11, 2022.
[38] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, Sep. 2009.
[39] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes challenge: A retrospective," *Int. J. Comput. Vis.*, vol. 111, no. 1, pp. 98–136, Jan. 2014.
[40] Y. Jiang et al., "R2CNN: Rotational region CNN for orientation robust scene text detection," 2017, *arXiv:1706.09579*.
[41] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[42] J. Yi, P. Wu, B. Liu, Q. Huang, H. Qu, and D. Metaxas, "Oriented object detection in aerial images with box boundary-aware vectors," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 2150–2159.
[43] J. Wang, W. Yang, H.-C. Li, H. Zhang, and G.-S. Xia, "Learning center probability map for detecting objects in aerial images," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 5, pp. 4307–4323, May 2021.
[44] J. Han, J. Ding, J. Li, and G.-S. Xia, "Align deep features for oriented object detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–11, 2022.
[45] W. Li, Y. Chen, K. Hu, and J. Zhu, "Oriented RepPoints for aerial object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 1829–1838.
[46] X. Yang et al., "Learning high-precision bounding box for rotated object detection via Kullback–Leibler divergence," 2021, *arXiv:2106.01883*.
[47] X. Yang, J. Yan, W. Liao, X. Yang, J. Tang, and T. He, "SCRDet++: Detecting small, cluttered and rotated objects via instance-level feature denoising and rotation loss smoothing," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, Apr. 12, 2022, doi: 10.1109/TPAMI.2022.3166956.
[48] Q. Ming, Z. Zhou, L. Miao, H. Zhang, and L. Li, "Dynamic anchor learning for arbitrary-oriented object detection," 2020, *arXiv:2012.04150*.
[49] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "*mixup*: Beyond empirical risk minimization," 2017, *arXiv:1710.09412*.
[50] X. Pan et al., "Dynamic refinement network for oriented and densely packed object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11207–11216.
[51] Q. Ming, L. Miao, Z. Zhou, X. Yang, and Y. Dong, "Optimization for arbitrary-oriented object detection via representation invariance loss," 2021, *arXiv:2103.11636*.
[52] P. Zhao, Z. Qu, Y. Bu, W. Tan, and Q. Guan, "PolarDet: A fast, more precise detector for rotated target in aerial images," *Int. J. Remote Sens.*, vol. 42, no. 15, pp. 5831–5861, Aug. 2021.
[53] X. Yang et al., "The KFIoU loss for rotated object detection," 2022, *arXiv:2201.12558*.
[54] X. Zhou, D. Wang, and P. Krähenbühl, "Objects as points," 2019, *arXiv:1904.07850*.
[55] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2021, pp. 10012–10022.
[56] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 4510–4520.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 10:26:07 UTC from IEEE Xplore. Restrictions apply.